WEEK 3 · DAY 1 · OVERVIEW

SQL With sqlite3



Created by **Jeff Lenamon**

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The Desk Lead's Note

The position file is moving into a database. Before any report comes out of it, give me the answers we already know, from SQL, each proved against pandas: the Energy bonds wider than 200 bp, the five largest positions, each sector's bonds, market value and average spread, the sectors holding at least 20 bonds, and the book's spread weighted by market value. Keep the queries where anyone can rerun them. Keep the database itself out of Git.

ABOUT 45 MIN WITH EXERCISES / 9 SECTIONS / 4 EXERCISES

Your Route Through Day 1

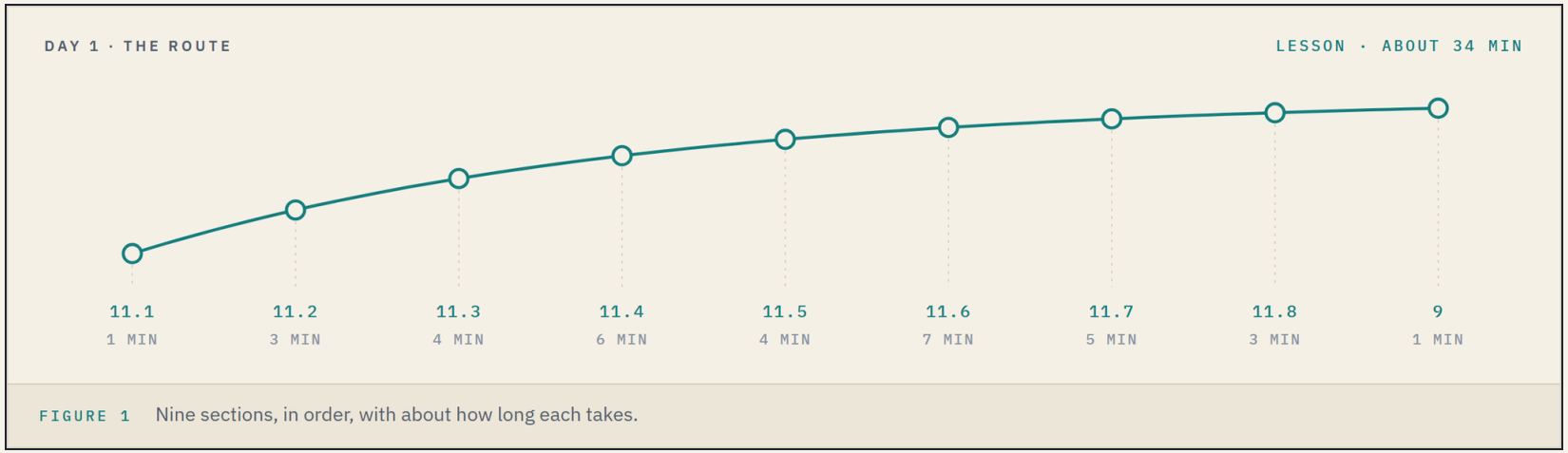


FIGURE 1 Nine sections, in order, with about how long each takes.

	SECTION	ABOUT
11.1	Why SQL, and Why Assistants Write So Much of It	1 min
11.2	**`sqlite3`**: a Database in One File	3 min
11.3	**`load_tables`**	4 min
11.4	**`SELECT`**, **`WHERE`**, **`ORDER BY`**, **`LIMIT`**, Each Beside pandas	6 min
11.5	Parameters With **`?`**	4 min

	SECTION	ABOUT
11.6	**`GROUP BY`** With **`COUNT`**, **`SUM`**, **`AVG`**, and **`HAVING`**	7 min
11.7	A Market-Value Weighted Average in SQL	5 min
11.8	Commit the Queries, Not the Database	3 min
9	Bring It Home	1 min

Time. About 34 minutes for the lesson, then about 12 for the four exercises.

11.1 Why SQL, and Why Assistants Write So Much of It

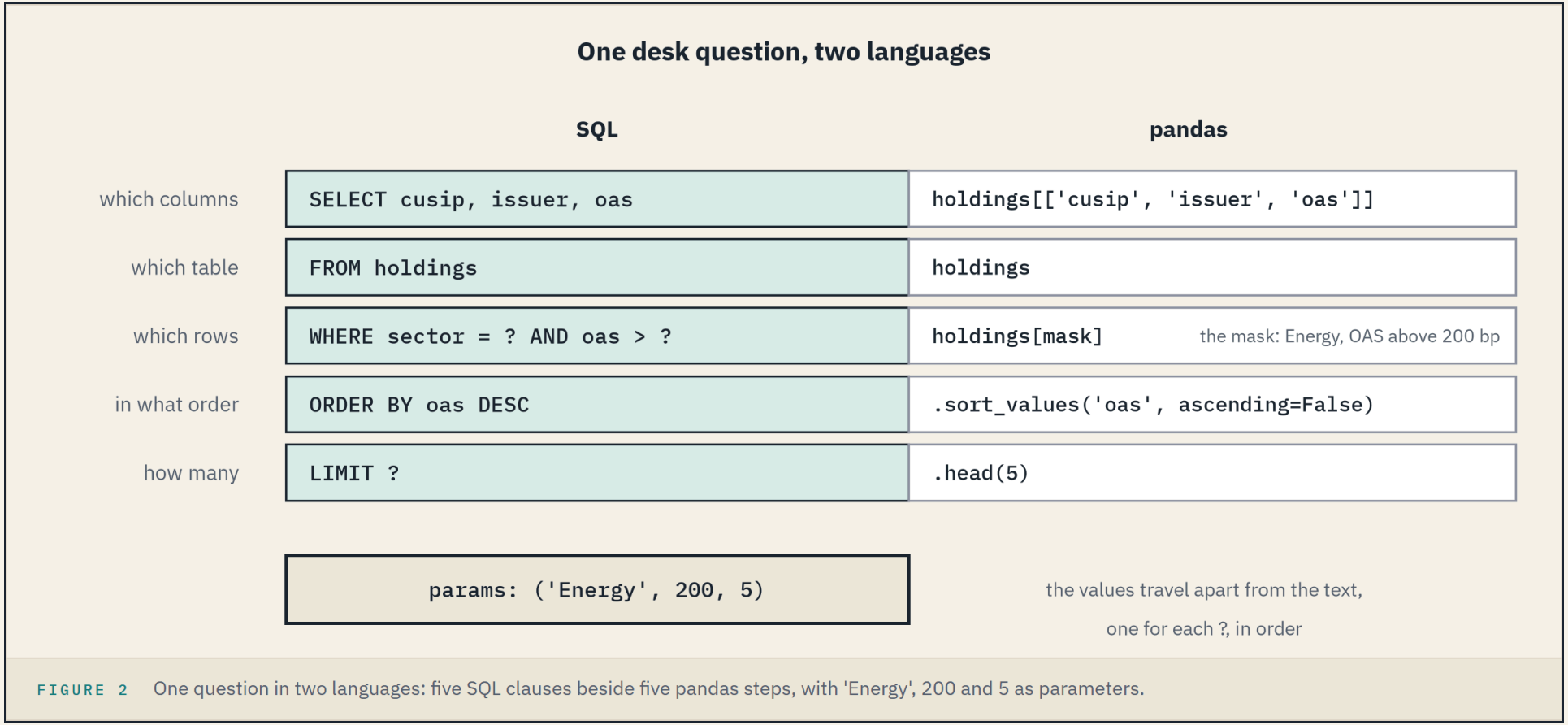


FIGURE 2 One question in two languages: five SQL clauses beside five pandas steps, with 'Energy', 200 and 5 as parameters.

SQL names the result you want, clause by clause; pandas is the answer key.

11.2 sqlite3: a Database in One File

TWO LINES LOAD A CSV

```
from contextlib import closing
db_path = work / 'course3.db'
with closing(sqlite3.connect(db_path)) as connection:
    holdings.to_sql('holdings', connection, if_exists='replace', index=False)
    count = connection.execute('SELECT COUNT(*) FROM holdings').fetchone()[0]
print(count, 'rows in the table,', len(holdings), 'in the file')
```

```
200 rows in the table, 200 in the file
```

IS IT TEXT?

```
print((work / 'holdings.csv').read_bytes()[:16])
print(db_path.read_bytes()[:16])
```

```
b'as_of_date,cusip'
b'SQLite format 3\x00'
```

WHICH SQLITE?

```
import sqlite3
print('SQLite', sqlite3.sqlite_version)
```

```
SQLite 3.50.4
```

Two lines put a CSV into a database file, and `COUNT(*)` reads its rows back: 200, the same as the file.

11.3 load_tables

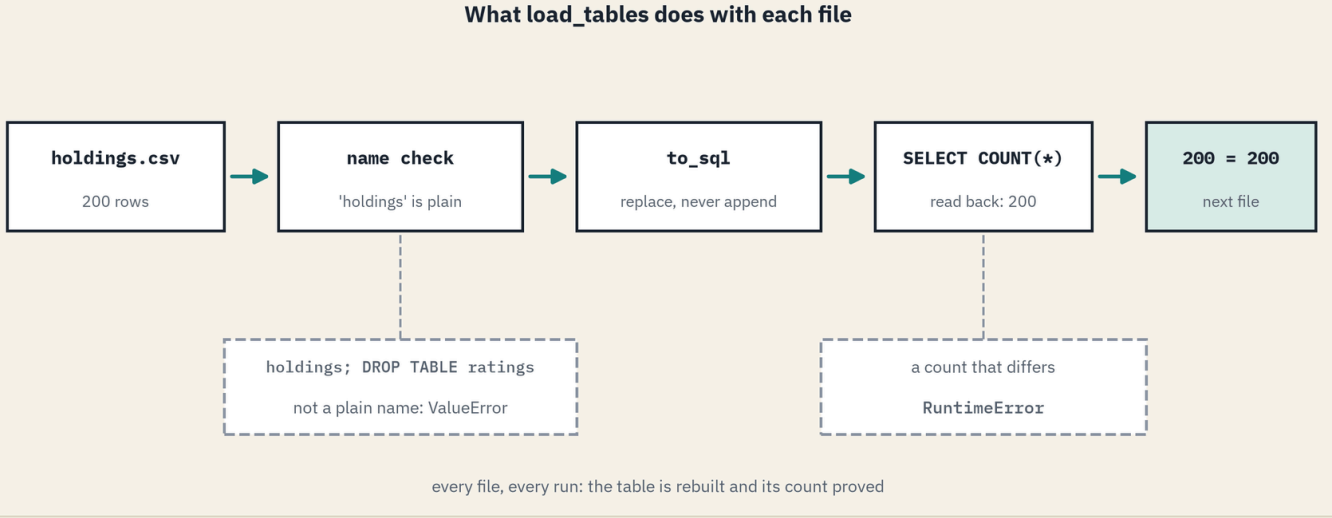


FIGURE 3 load_tables checks each name, writes the table, reads the count back, and raises if 200 is not 200.

FIVE TABLES

```
table_counts = md.load_tables(db_path, TABLES)
table_counts
```

```
{'holdings': 200,
 'issuers': 150,
 'ratings': 18,
 'benchmark': 821,
 'trades': 374}
```

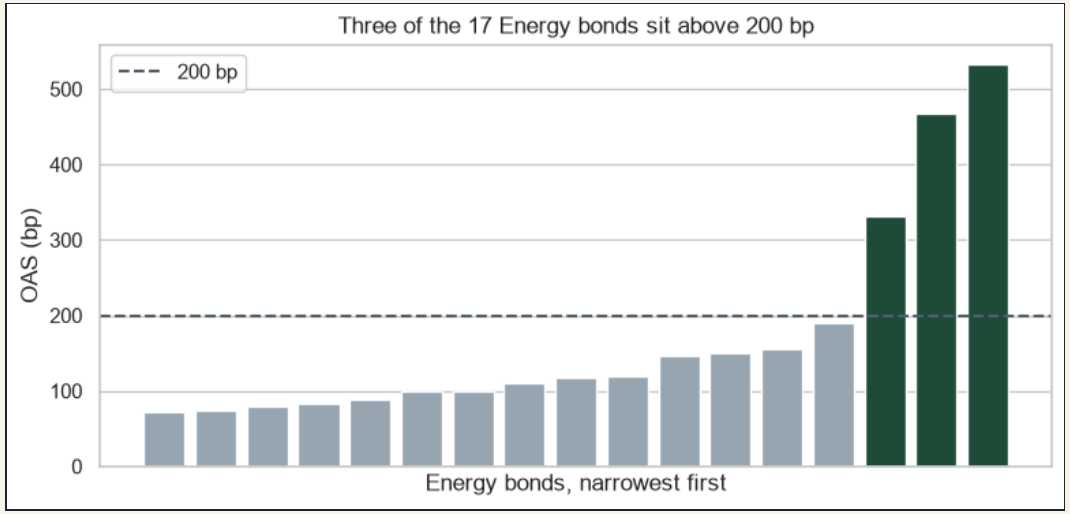
`load_tables` **writes each CSV to a table and refuses to finish unless every count matches its file.**

11.4 SELECT, WHERE, ORDER BY, LIMIT, Each Beside pandas

WHICH ENERGY BONDS ARE WIDER THAN 200 BP?

```
sql = 'SELECT cusip, issuer, oas FROM holdings WHERE sector = ? AND oas > ?'
from_sql = md.run_query(db_path, sql, ('Energy', 200))
mask = (holdings['sector'] == 'Energy') & (holdings['oas'] > 200)
```

```
cusip          issuer  oas
0  99145PAA0  Praximer Energy  468
1  99074WAB8  Wyxundra Energy  532
2  99074WAD4  Wyxundra Energy  332
```



EXCEL SIDE BY SIDE

JOB	EXCEL
Energy wider than 200 bp	`=FILTER(A2:A201,(C2:C201="Energy")*(E2:E201>200))`

11.5 Parameters With ?

PASTED INTO THE SQL TEXT

```
ticker = "O'BRN"
pasted = md.run_query(db_path, f"SELECT cusip FROM holdings WHERE ticker = '{ticker}'")
```

```
DatabaseError: Execution failed on sql 'SELECT cusip FROM holdings WHERE ticker = 'O'BRN'': near "BRN": syntax error
```

SENT AS A PARAMETER

```
as_parameter = md.run_query(db_path, 'SELECT cusip FROM holdings WHERE ticker = ?', (ticker,))
print(len(as_parameter), 'bonds, and no error')
```

```
0 bonds, and no error
```

The quote in `O'BRN` closed the SQL text early, and SQLite could not read the rest.

11.6 GROUP BY With COUNT, SUM, AVG, and HAVING

ONE ROW PER SECTOR

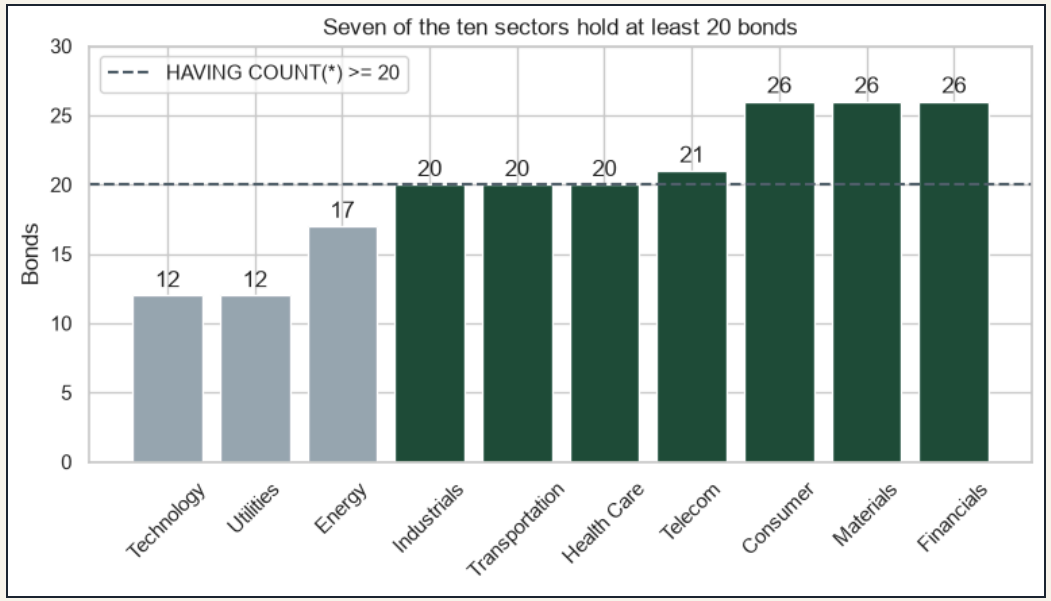
```
sql = '''SELECT sector, COUNT(*) AS bonds, SUM(market_value) AS market_value, AVG(oas) AS avg_oas
         FROM holdings GROUP BY sector ORDER BY market_value DESC'''
by_sector_sql = md.run_query(db_path, sql)
by_sector_sql.style.hide(axis='index').format(
    {'market_value': '{:,.2f}', 'avg_oas': '{:.2f}'}).bar(subset=['market_value'], color='#9AA5B1')
```

SECTOR	BONDS	MARKET_VALUE	AVG_OAS
Materials	26	71,701,617.50	182.42
Financials	26	68,219,672.50	172.31
Consumer	26	56,629,597.50	218.88
Telecom	21	55,848,270.00	182.38
Industrials	20	52,540,745.00	220.55

EXCEL SIDE BY SIDE

JOB	EXCEL
One row per sector	the pivot table
Bonds per sector	`=COUNTIFS($C$2:$C$201,J2)`
Market value	`=SUMIFS($F$2:$F$201,$C$2:$C$201,J2)`
Average OAS	`=AVERAGEIFS($E$2:$E$201,$C$2:$C$201,J2)`

HAVING Keeps Whole Groups



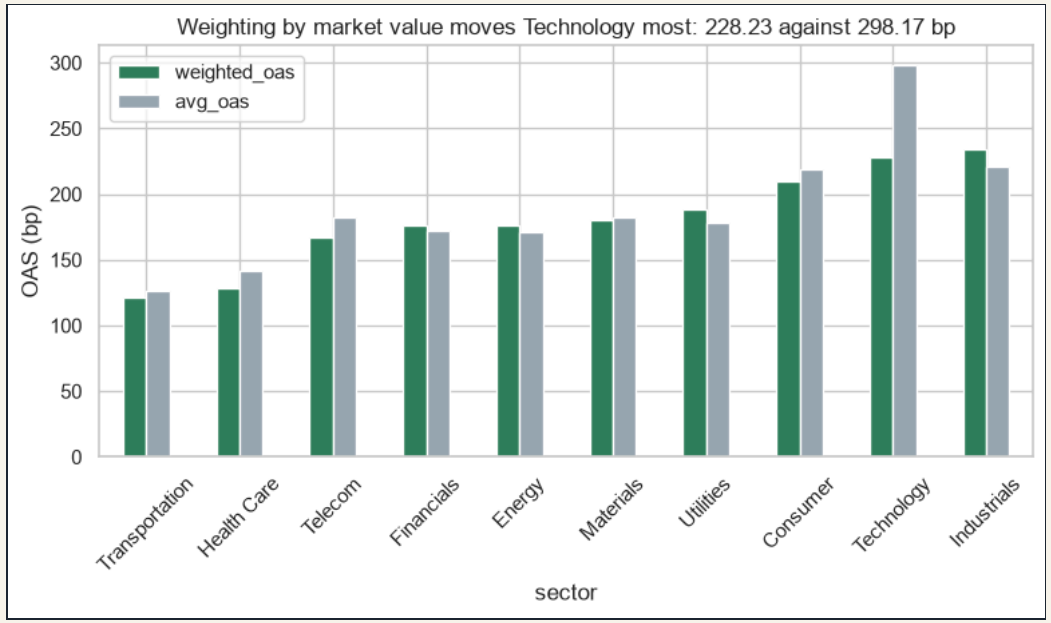
HAVING TESTS EACH SECTOR'S COUNT

```
sql = '''SELECT sector, COUNT(*) AS bonds FROM holdings
         GROUP BY sector HAVING COUNT(*) >= ? ORDER BY sector'''
kept = md.run_query(db_path, sql, (20,))
counts = by_sector_pandas.reset_index()[['sector', 'bonds']]
md.assert_same_rows(kept, counts[counts['bonds'] >= 20], sort_by=['sector'])
print(len(kept), 'of', len(counts), 'sectors:', ', '.join(kept['sector']))
```

EXCEL SIDE BY SIDE

`=FILTER(J2:K11,K2:K11>=20)` on the summary block, its counts in K, returns the same 7 sectors.

11.7 A Market-Value Weighted Average in SQL



THE BOOK IN ONE ROW

```
sql = '''SELECT SUM(oas * market_value) / SUM(market_value) AS weighted_oas,
                AVG(oas) AS avg_oas
         FROM holdings'''
```

```
SQL 180.4609519769922 bp, bondmath 180.4609519769922 bp
simple average 185.75 bp
```

EXCEL SIDE BY SIDE

JOB	RESULT
Weighted OAS by sector	Energy 176.34941368150737, all ten equal

180.46 bp is the spread on Course 2's risk card, in Portfolio Aggregation and Scenarios.

11.8 Commit the Queries, Not the Database

IS THE DATABASE IGNORED?

```
!git -C "{work}" check-ignore -v course3.db
```

```
.gitignore:5:*.db       course3.db
```

COMMIT THREE FILES BY NAME

```
!git -C "{work}" add marketdata.py test_marketdata.py queries.sql
!git -C "{work}" commit -m "Notebook 11: load_tables, run_query, assert_same_rows, and queries.sql"
!git -C "{work}" show --stat --oneline HEAD
```

```
[main fb9ff0f] Notebook 11: load_tables, run_query, assert_same_rows, and queries.sql
 3 files changed, 134 insertions(+)
 create mode 100644 queries.sql
```

The record holds the code, the tests and the queries; one call to `load_tables` rebuilds the database, so it is never committed.

Rebuild the day's work in your own folder.

Bring It Home

IN A POWERSHELL TERMINAL

```
# the work folder: use the name your setup cell printed
$work = "$env:TEMP\pfi_course3_11_k4w7q2xr"
Set-Location "$HOME\projects\my_bondmath"
Copy-Item "$work\marketdata.py", "$work\test_marketdata.py", "$work\queries.sql" .
Copy-Item "$work\holdings.csv", "$work\ratings.csv" .
python -m pytest -q test_marketdata.py
git status --short
git add marketdata.py test_marketdata.py queries.sql
git commit -m "Notebook 11: load_tables, run_query, assert_same_rows, and queries.sql"
```

pytest shows 46 passed.

You Can Now

- Put CSV files into a SQLite database file with **`load_tables`**, every row count checked.
- Ask with **`SELECT`**, **`WHERE`**, **`ORDER BY`** and **`LIMIT`**, and prove each answer against pandas with **`assert_same_rows`**.
- Pass every value as a **`?`** parameter, and say what SQL injection is.
- Summarise with **`GROUP BY`**, **`COUNT`**, **`SUM`** and **`AVG`**, and filter groups with **`HAVING`**.
- Write a weighted average in SQL, and catch a division of whole numbers with a twin.
- Commit the queries and keep the database out of Git.

NEXT
WEEK 3 · DAY 2

SQL Joins and Time Series

Joins across the book's tables, each counted before and after against a control figure, and the 10-year yield as a time series in SQL, with LAG and ROW_NUMBER proved against your own functions.

